



Educación Vial IMSJ

Flujo del backend PHP nativo

42 endpoints · Código comentado · Referencia API simple

Versión de trabajo | 05/10/2026 | CC-05

Documento	Estado
Implementación	PHP 8.5, PDO y MySQL 8.4; siete módulos.
Revisión interna	Juan Robaina; pendiente.
Aceptación	No registrada por esta intervención.
Autoría y responsabilidad	Tomás Cabrera, Juan Robaina, Gabriela Romero, Verónica Romero y Juan Corrales.

Preparado con apoyo externo de IA. Las explicaciones describen el código de esta versión; los ejemplos son didácticos. No reconstruyen reuniones ni acreditan aceptación del cliente o de docentes.

Agenda y reservas: alcance académico. CC-01 a CC-04: pendientes para la entrega real, sin fecha ni prioridad distinta.

Cómo recorre una petición la API

La entrada única lee método y ruta. El switch llama al controlador indicado. El controlador exige el acceso, valida la entrada y llama al servicio. El servicio aplica reglas y coordina transacciones; el repositorio accede a MySQL por PDO preparado. El modelo presenta datos y Response entrega JSON o 204.

Pieza	Responsabilidad
public/index.php	Carga explícita, CORS, switch y manejo final de errores.
Controlador	Validación HTTP y Auth::requireUser por operación.
Servicio	Reglas del módulo; transacción y ciclo de archivos.
Repositorio	SQL parametrizado; SELECT/INSERT/UPDATE/DELETE.
Database	PDO; begin/commit/rollback antes de responder.
Modelo / Response	Claves JSON conservadas y código HTTP.

Los siete módulos

Auth · Noticia · Material · Pregunta · Prueba · Reserva · Usuario. Reserva incluye franjas y agenda; Usuario incluye historial. Cada módulo tiene un controlador, servicio y repositorio.

Cómo usar las fichas

Las páginas 4 a 24 contienen dos fichas por página, en el orden del switch. Cada ficha muestra entrada, acceso, métodos, tablas, salida y efectos. Salud evita SQL; perfil valida token directamente; las demás fichas señalan el servicio utilizado.

Lectura junto al código

Buscar método/ruta en public/index.php; abrir el método de controlador, seguir el método de servicio y después el SQL del repositorio. Los comentarios explican validación, autenticación, transacción y limpieza. La versión Markdown contiene las mismas 42 fichas.

Sesiones, archivos y respuestas

Bearer revocable

El token tiene forma id|secreto. El secreto usa 32 bytes aleatorios; SQL conserva SHA-256. Dura ocho horas. Cada petición verifica vencimiento, cuenta activa y rol. Nuevo login revoca anteriores; logout/desactivación revocan. Sesiones anteriores a la reescritura: volver a ingresar.

Formularios y adjuntos

JSON para datos; multipart para archivos. El panel edita mediante POST con `_method=PUT`; también se admite PUT multipart nativo. MIME y contenido comprobados, nombres aleatorios y almacenamiento fuera de public. `/storage` sirve solo PDF/JPG/PNG/WEBP permitidos; el formulario global admite 24 MB.

Código HTTP	Uso
200 / 201	Consulta, edición o acción / alta, registro o reserva.
204	Eliminación, desactivación o logout sin cuerpo.
400 / 413 / 422	JSON inválido / formulario demasiado grande / validación o conflicto.
401 / 403 / 404	Sesión inválida / rol insuficiente / ruta o recurso inexistente.
429 / 500	Límite de intentos / error interno sin revelar SQL o secretos.

Contrato de los clientes

Se mantienen claves raíz como noticias, material, preguntas, reservas y usuario; errores message y errors. No se introduce data/datos. Los frontends mantienen sus archivos y direcciones. Los errores 500 son posibles en todas las fichas ante un fallo inesperado.

Fechas y valores del código

Fechas civiles AAAA-MM-DD en Uruguay; timestamps SQL UTC. Cantidades del test, límites de archivos y clave inicial describen comportamiento conservado y no acreditan una decisión institucional nueva.

Endpoints 01-02

01 GET /api/health

Comprobar que la entrada PHP responde.

Acceso: Público: sin token.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php identifica GET health y llama directamente a Response::json. No hay controlador, servicio, repositorio ni consulta SQL.

Persistencia: Sin consulta de base de datos.

Respuesta: 200, {"status":"ok"}.

Reglas y efectos: No acredita que MySQL esté conectado o que el sistema esté listo para operar.

Errores: 404 si método/ruta no coinciden. El health no detecta fallos de MySQL.

02 POST /api/login

Iniciar sesión conservando cédula, Bearer y perfil usados por ambos frontends.

Acceso: Público: sin token.

Entrada: JSON: cedula (7 u 8 dígitos, admite separadores) y password. Se conserva el secreto sin recortar espacios; hasta 72 bytes por bcrypt.

Recorrido: public/index.php → AuthController::login → Auth::limitAttempts(login) → validación → AuthService::login → UsuarioRepository::buscarCedula/bloquear → password_verify → AuthRepository::revocarTodos/crear → Response.

Persistencia: usuarios y auth_tokens

Respuesta: 200, {token, expira_en, usuario:{id,nombre,cedula,rol}}.

Reglas y efectos: Transacción: emitir un secreto aleatorio de 32 bytes, guardar solo SHA-256 y revocar sesiones anteriores. Vigencia de ocho horas, vencimiento ISO 8601.

Errores: 400 JSON mal formado; 422 credenciales inválidas/cuenta inactiva; 429 más de cinco intentos en 60 segundos; 500 SQL.

Endpoints 03-04

03 POST /api/register

Crear una cuenta ciudadana y emitir su primera sesión.

Acceso: Público: sin token.

Entrada: JSON: cedula, password (mínimo 6 caracteres y máximo 72 bytes), password_confirmation idéntico. Un rol enviado por el cliente no modifica el rol ciudadano.

Recorrido: public/index.php → AuthController::register → límite de intentos/validación → AuthService::register → UsuarioRepository::buscarCedula/crear → password_hash bcrypt → AuthRepository::revocarTodos/crear → Response.

Persistencia: usuarios y auth_tokens

Respuesta: 201, {token, expira_en, usuario}; nombre puede ser null y rol PUBLICO_GENERAL.

Reglas y efectos: Alta y sesión en la misma transacción. La cédula es única. Cinco intentos/minuto por IP para registro, separado del contador de login.

Errores: 400 JSON inválido; 422 cédula repetida o datos/confirmación inválidos; 429 límite; 500 SQL.

04 GET /api/portal/noticias

Listar contenidos visibles para la ciudadanía.

Acceso: Público: sin token.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → NoticiaController::publicIndex → acceso/validación → NoticiaService::listar → NoticiaRepository::listar → Noticia::toArray/Response.

Persistencia: noticias, noticia_imagenes y noticia_enlaces

Respuesta: 200, {noticias:[...]}.

Reglas y efectos: Solo PUBLICADO; noticias además dentro de vigencia inclusiva.

Errores: 500 ante un fallo interno o de MySQL, salvo health.

Endpoints 05-06

05 GET /api/portal/materiales

Listar contenidos visibles para la ciudadanía.

Acceso: Público: sin token.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → MaterialController::publicIndex → acceso/validación → MaterialService::listar → MaterialRepository::listar → Material::toArray/Response.

Persistencia: materiales_estudio

Respuesta: 200, {materiales:[...]}.

Reglas y efectos: Solo PUBLICADO; noticias además dentro de vigencia inclusiva.

Errores: 500 ante un fallo interno o de MySQL, salvo health.

06 GET /api/portal/preguntas

Listar contenidos visibles para la ciudadanía.

Acceso: Público: sin token.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → PreguntaController::publicIndex → acceso/validación → PreguntaService::listar → PreguntaRepository::listar → Pregunta::toArray/Response.

Persistencia: preguntas_frecuentes

Respuesta: 200, {preguntas:[...]}.

Reglas y efectos: Solo PUBLICADO; noticias además dentro de vigencia inclusiva.

Errores: 500 ante un fallo interno o de MySQL, salvo health.

Endpoints 07-08

07 GET /api/portal/prueba

Obtener el cuestionario público sin revelar respuestas correctas.

Acceso: Público: sin token.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → PruebaController::publicIndex → PruebaService::publica → PruebaRepository::aleatorias → SELECT ORDER BY RAND() LIMIT 10 → Prueba::toArray(false) → Response.

Persistencia: preguntas_prueba

Respuesta: 200, {preguntas:[{id,pregunta,opciones:{A,B,C,D}},...]}.

Reglas y efectos: Hasta diez preguntas según el banco existente; no acredita criterios acordados para cerrar CC-01.

Errores: 500 ante un fallo interno o de MySQL, salvo health.

08 POST /api/portal/prueba/corregir

Corregir las respuestas contra el banco almacenado.

Acceso: Público: sin token.

Entrada: JSON: respuestas, lista de 1-20 objetos {pregunta_id, opcion}. Identificadores distintos, existentes y opciones A/B/C/D.

Recorrido: public/index.php → PruebaController::corregir → validación de lista → PruebaService::corregir → PruebaRepository::buscar por pregunta → comparación con respuesta_correcta SQL → Response.

Persistencia: preguntas_prueba

Respuesta: 200, {total,correctas,resultados:[{pregunta_id,correcta,respuesta_correcta},...]}.

Reglas y efectos: No guarda un resultado personal ni acepta un campo enviado como respuesta correcta. No agrega duración, nota de aprobación o adjuntos.

Errores: 400 JSON inválido; 422 lista/identificador/opción inválidos o pregunta repetida; 500 SQL.

Endpoints 09-10

09 GET /api/franjas/disponibles

Consultar disponibilidad pública futura con cupos.

Acceso: Público: sin token.

Entrada: Query tipo opcional con uno de los tres tipos de trámite.

Recorrido: public/index.php → ReservaController::disponibles → acceso/validación → ReservaService::franjas → ReservaRepository::franjas → Franja/Reserva → Response.

Persistencia: franjas_disponibilidad y reservas

Respuesta: 200, {franjas:[{id,fecha,hora_inicio,hora_fin,tipo,cupos_totales,cupos_disponibles},...]}.

Reglas y efectos: El repositorio cuenta reservas por franja; la consulta pública filtra fecha desde hoy y cupos disponibles positivos.

Errores: 500 ante un fallo interno o de MySQL, salvo health. 422 si el filtro de tipo es inválido.

10 GET /api/me

Consultar el perfil de la cuenta que presenta el token.

Acceso: Bearer válido de cualquier rol, cuenta activa.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → AuthController::me → Auth::requireUser → AuthRepository::buscarToken → UsuarioRepository::buscar → Usuario::toArray → Response. No interviene AuthService.

Persistencia: usuarios y auth_tokens

Respuesta: 200, {usuario:{id,nombre,cedula,rol}}; sin hash o datos internos del token.

Reglas y efectos: Se revalidan hash, vencimiento y cuenta activa; el perfil no se toma del sessionStorage del navegador.

Errores: 401 si la sesión es inválida.

Endpoints 11-12

11 POST /api/logout

Revocar la sesión utilizada en la petición.

Acceso: Bearer válido de cualquier rol, cuenta activa.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → AuthController::logout → Auth::requireUser → AuthService::logout → AuthRepository::revocar(tokenId) → DELETE auth_tokens → Response::empty.

Persistencia: usuarios y auth_tokens

Respuesta: 204 sin cuerpo.

Reglas y efectos: Después del cierre, el mismo Bearer devuelve 401. Borrar la sesión del navegador por sí solo no revoca el token del servidor.

Errores: 401 si la sesión es inválida.

12 POST /api/reservas

Reservar un cupo académico para la cuenta ciudadana autenticada.

Acceso: Bearer válido, PUBLICO_GENERAL y cuenta activa.

Entrada: JSON: franja_disponibilidad_id, entero positivo. El usuario_id se obtiene del token y no del cuerpo.

Recorrido: public/index.php → ReservaController::store → acceso/validación → ReservaService::reservar → ReservaRepository::franja con bloqueo/duplicada/crearReserva → Franja/Reserva → Response.

Persistencia: franjas_disponibilidad y reservas

Respuesta: 201,
{reserva:{id,reserva_id,franja_disponibilidad_id,fecha,hora_inicio,hora_fin,tipo,tipo_tramite,creada_en}}.

Reglas y efectos: Transacción: SELECT FOR UPDATE en franja, comprobar existencia/fecha/duplicación/cupo, insertar y commit. Si dos procesos disputan el último cupo, solo uno reserva.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 422 id inexistente, fecha vencida, duplicación o falta de cupos.

Endpoints 13-14

13 GET /api/reservas/mias

Listar únicamente las reservas de la cuenta ciudadana autenticada.

Acceso: Bearer válido, PUBLICO_GENERAL y cuenta activa.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → ReservaController::mine → acceso/validación → ReservaService::propias → ReservaRepository::propias → Franja/Reserva → Response.

Persistencia: franjas_disponibilidad y reservas

Respuesta: 200, {reservas:[...]}, con fechas/horas/tipo y creada_en.

Reglas y efectos: El filtro usuario_id proviene del token; no se acepta seleccionar otra persona en query/body.

Errores: 401 sin sesión válida; 403 con rol insuficiente.

14 GET /api/historial

Consultar las acciones administrativas más recientes.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: Query limite opcional, entero 1-50 (por defecto 20).

Recorrido: public/index.php → UsuarioController::historial → Auth/validación → UsuarioService::historial → UsuarioRepository::historial → SELECT con usuario y LIMIT entero → Response.

Persistencia: usuarios e historial_acciones

Respuesta: 200, {acciones:[...]} con usuario, acción, tipo de elemento, id afectado y fecha.

Reglas y efectos: Solo lectura; una eliminación conserva su referencia en historial aunque el contenido ya no exista.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 422 límite no válido.

Endpoints 15-16

15 GET /api/usuarios-admin

Listar integrantes del personal que conservan acceso activo.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → UsuarioController::index → Auth::requireUser(personal) → UsuarioService::listar → UsuarioRepository::personal → Usuario/Response.

Persistencia: usuarios e historial_acciones

Respuesta: 200, {usuarios:[{id,nombre,cedula,created_at},...]}.

Reglas y efectos: Filtra PERSONAL_IMSJ y activo=1; no expone contraseñas o tokens.

Errores: 401 sin sesión válida; 403 con rol insuficiente.

16 POST /api/usuarios-admin

Crear o reactivar una cuenta de personal desde el panel.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: JSON: nombre no vacío (≤ 120) y cedula normalizada de 7 u 8 dígitos.

Recorrido: public/index.php → UsuarioController::store → Auth::requireUser(personal) → UsuarioService::crear → UsuarioRepository::bloquearPersonal/buscarCedula/crear o reactivar/registrarse → Usuario/Response.

Persistencia: usuarios e historial_acciones

Respuesta: 201, {usuario:{id,nombre,cedula}, clave_inicial:"imsj1234"}.

Reglas y efectos: Conserva la clave inicial del comportamiento anterior. Transacción con auditoría; no convierte una cuenta ciudadana en personal. Reactivar revoca sesiones anteriores.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 422 si cédula pertenece a ciudadanía o personal ya activo.

Endpoints 17-18

17 DELETE /api/usuarios-admin/{id}

Desactivar personal sin eliminar su historia o reservas.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → UsuarioController::destroy → Auth::requireUser(personal) → UsuarioService::desactivar → UsuarioRepository::bloquearPersonal/buscar/desactivar/regar → Usuario/Response.

Persistencia: usuarios e historial_acciones

Respuesta: 204 sin cuerpo.

Reglas y efectos: Transacción: exige cuenta de personal activa, impide quitar acceso propio/último integrante, revoca sus tokens y conserva referencias históricas.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 404 integrante activo inexistente; 422 acceso propio/último personal.

18 GET /api/preguntas-prueba

Listar todos los elementos del módulo para el personal.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → PruebaController::index → acceso/validación → PruebaService::listar → PruebaRepository::listar → Prueba::toArray/Response.

Persistencia: preguntas_prueba

Respuesta: 200, {preguntas:[...]}.

Reglas y efectos: Banco completo con la respuesta correcta para el personal; no modifica preguntas.

Errores: 401 sin sesión válida; 403 con rol insuficiente.

Endpoints 19-20

19 POST /api/preguntas-prueba

Crear un elemento del módulo.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: pregunta (≤ 500), opcion_a a opcion_d (≤ 255 cada una) y respuesta_correcta A/B/C/D.

Recorrido: public/index.php → PruebaController::store → acceso/validación → PruebaService::guardar → PruebaRepository::crear → Prueba::toArray/Response.

Persistencia: preguntas_prueba; historial_acciones (UsuarioRepository::registrar)

Respuesta: 201, {pregunta:{...}}.

Reglas y efectos: Transacción de datos más auditoría. Banco de preguntas base; no incorpora gráficos de CC-04.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 400 JSON inválido; 422 datos/archivos inválidos; 404 id inexistente en edición.

20 PUT /api/preguntas-prueba/{id}

Editar los datos del elemento existente.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: pregunta (≤ 500), opcion_a a opcion_d (≤ 255 cada una) y respuesta_correcta A/B/C/D.

Recorrido: public/index.php → PruebaController::update → acceso/validación → PruebaService::guardar → PruebaRepository::buscar/actualizar → Prueba::toArray/Response.

Persistencia: preguntas_prueba; historial_acciones (UsuarioRepository::registrar)

Respuesta: 200, {pregunta:{...}}.

Reglas y efectos: Transacción de datos más auditoría. Banco de preguntas base; no incorpora gráficos de CC-04.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 400 JSON inválido; 422 datos/archivos inválidos; 404 id inexistente en edición.

Endpoints 21-22

21 DELETE /api/preguntas-prueba/{id}

Eliminar un elemento y mantener constancia en historial.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → PruebaController::destroy → acceso/validación → PruebaService::eliminar → PruebaRepository::buscar/eliminar → Prueba::toArray/Response.

Persistencia: preguntas_prueba; historial_acciones (UsuarioRepository::registrar)

Respuesta: 204 sin cuerpo.

Reglas y efectos: El servicio registra ELIMINAR y borra en la misma transacción. No elimina el registro de auditoría.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 404 id inexistente.

22 GET /api/noticias

Listar todos los elementos del módulo para el personal.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → NoticiaController::index → acceso/validación → NoticiaService::listar → NoticiaRepository::listar → Noticia::toArray/Response.

Persistencia: noticias, noticia_imagenes y noticia_enlaces

Respuesta: 200, {noticias:[...]}.

Reglas y efectos: Incluye elementos no publicados; no altera su estado.

Errores: 401 sin sesión válida; 403 con rol insuficiente.

Endpoints 23-24

23 POST /api/noticias

Crear un elemento del módulo.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: titulo (≤ 255), texto, fecha_inicio_vigencia y fecha_fin_vigencia (AAAA-MM-DD). Opcionales: imagen_portada, galeria[] (≤ 5), enlaces[] (≤ 5 HTTP/HTTPS). Imágenes JPG/PNG/WEBP ≤ 2048 KB cada una.

Recorrido: public/index.php → NoticiaController::store → acceso/validación → NoticiaService::guardar → NoticiaRepository::crear → Noticia::toArray/Response.

Persistencia: noticias, noticia_imagenes y noticia_enlaces; historial_acciones (UsuarioRepository::registrar)

Respuesta: 201, {noticia:{...}}.

Reglas y efectos: Transacción de datos más auditoría. Comienza NO_PUBLICADO. Archivos nuevos antes de SQL, limpieza si falla; anteriores después del commit. En edición, enlaces ausentes se conservan y [] los elimina; galería se reemplaza al enviar imágenes.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 400 JSON inválido; 422 datos/archivos inválidos; 404 id inexistente en edición.

24 GET /api/noticias/{id}

Consultar el detalle administrativo de una noticia.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → NoticiaController::show → acceso/validación → NoticiaService::detalle → NoticiaRepository::buscar → Noticia::toArray/Response.

Persistencia: noticias, noticia_imagenes y noticia_enlaces

Respuesta: 200, {noticia:{...}}, con imágenes/enlaces y fechas de vigencia.

Reglas y efectos: No modifica datos ni registra una acción administrativa.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 404 noticia inexistente.

Endpoints 25-26

25 PUT /api/noticias/{id}

Editar los datos del elemento existente.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: titulo (≤ 255), texto, fecha_inicio_vigencia y fecha_fin_vigencia (AAAA-MM-DD). Opcionales: imagen_portada, galeria[] (≤ 5), enlaces[] (≤ 5 HTTP/HTTPS). Imágenes JPG/PNG/WEBP ≤ 2048 KB cada una.

Recorrido: public/index.php → NoticiaController::update → acceso/validación → NoticiaService::guardar → NoticiaRepository::buscar/actualizar → Noticia::toArray/Response.

Persistencia: noticias, noticia_imagenes y noticia_enlaces; historial_acciones (UsuarioRepository::registrar)

Respuesta: 200, {noticia:{...}}.

Reglas y efectos: Transacción de datos más auditoría. Conserva el estado de publicación existente. Archivos nuevos antes de SQL, limpieza si falla; anteriores después del commit. En edición, enlaces ausentes se conservan y [] los elimina; galería se reemplaza al enviar imágenes.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 400 JSON inválido; 422 datos/archivos inválidos; 404 id inexistente en edición.

26 PATCH /api/noticias/{id}/estado

Publicar o retirar el contenido mediante una operación separada.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: JSON: estado=PUBLICADO o NO_PUBLICADO.

Recorrido: public/index.php → NoticiaController::updateEstado → acceso/validación → NoticiaService::estado → NoticiaRepository::buscar/actualizar → Noticia::toArray/Response.

Persistencia: noticias, noticia_imagenes y noticia_enlaces; historial_acciones (UsuarioRepository::registrar)

Respuesta: 200, {noticia:{...}}.

Reglas y efectos: Si cambia el estado, persiste y audita PUBLICAR/DESPUBLICAR en una transacción. Si ya tiene ese estado devuelve el elemento sin duplicar acción. No implementa aprobación de Directora.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 422 estado inválido; 404 id inexistente.

Endpoints 27-28

27 DELETE /api/noticias/{id}

Eliminar un elemento y mantener constancia en historial.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → NoticiaController::destroy → acceso/validación → NoticiaService::eliminar → NoticiaRepository::buscar/eliminar → Noticia::toArray/Response.

Persistencia: noticias, noticia_imagenes y noticia_enlaces; historial_acciones (UsuarioRepository::registrar)

Respuesta: 204 sin cuerpo.

Reglas y efectos: El servicio registra ELIMINAR y borra en la misma transacción. Noticia elimina galería/enlaces en cascada; limpia sus archivos después del commit.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 404 id inexistente.

28 GET /api/materiales

Listar todos los elementos del módulo para el personal.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → MaterialController::index → acceso/validación → MaterialService::listar → MaterialRepository::listar → Material::toArray/Response.

Persistencia: materiales_estudio

Respuesta: 200, {materiales:[...]}.

Reglas y efectos: Incluye elementos no publicados; no altera su estado.

Errores: 401 sin sesión válida; 403 con rol insuficiente.

Endpoints 29-30

29 POST /api/materiales

Crear un elemento del módulo.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: nombre (≤ 255), tipo PDF/IMAGEN/VIDEO. archivo para PDF (≤ 10240 KB) o imagen (≤ 5120 KB); ubicacion_recurso HTTP/HTTPS (≤ 2048) para video. El tipo nuevo exige recurso.

Recorrido: public/index.php → MaterialController::store → acceso/validación → MaterialService::guardar → MaterialRepository::crear → Material::toArray/Response.

Persistencia: materiales_estudio; historial_acciones (UsuarioRepository::registrar)

Respuesta: 201, {material:{...}}.

Reglas y efectos: Transacción de datos más auditoría. Comienza NO_PUBLICADO. Nuevo tipo requiere recurso; mismo tipo permite conservarlo. Se limpia archivo nuevo ante fallo y anterior tras confirmar reemplazo.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 400 JSON inválido; 422 datos/archivos inválidos; 404 id inexistente en edición.

30 PUT /api/materiales/{id}

Editar los datos del elemento existente.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: nombre (≤ 255), tipo PDF/IMAGEN/VIDEO. archivo para PDF (≤ 10240 KB) o imagen (≤ 5120 KB); ubicacion_recurso HTTP/HTTPS (≤ 2048) para video. El tipo nuevo exige recurso.

Recorrido: public/index.php → MaterialController::update → acceso/validación → MaterialService::guardar → MaterialRepository::buscar/actualizar → Material::toArray/Response.

Persistencia: materiales_estudio; historial_acciones (UsuarioRepository::registrar)

Respuesta: 200, {material:{...}}.

Reglas y efectos: Transacción de datos más auditoría. Conserva el estado de publicación existente. Nuevo tipo requiere recurso; mismo tipo permite conservarlo. Se limpia archivo nuevo ante fallo y anterior tras confirmar reemplazo.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 400 JSON inválido; 422 datos/archivos inválidos; 404 id inexistente en edición.

Endpoints 31-32

31 PATCH /api/materiales/{id}/estado

Publicar o retirar el contenido mediante una operación separada.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: JSON: estado=PUBLICADO o NO_PUBLICADO.

Recorrido: public/index.php → MaterialController::updateEstado → acceso/validación → MaterialService::estado → MaterialRepository::buscar/actualizar → Material::toArray/Response.

Persistencia: materiales_estudio; historial_acciones (UsuarioRepository::registrar)

Respuesta: 200, {material:{...}}.

Reglas y efectos: Si cambia el estado, persiste y audita PUBLICAR/DESPUBLICAR en una transacción. Si ya tiene ese estado devuelve el elemento sin duplicar acción. No implementa aprobación de Directora.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 422 estado inválido; 404 id inexistente.

32 DELETE /api/materiales/{id}

Eliminar un elemento y mantener constancia en historial.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → MaterialController::destroy → acceso/validación → MaterialService::eliminar → MaterialRepository::buscar/eliminar → Material::toArray/Response.

Persistencia: materiales_estudio; historial_acciones (UsuarioRepository::registrar)

Respuesta: 204 sin cuerpo.

Reglas y efectos: El servicio registra ELIMINAR y borra en la misma transacción. Material limpia su archivo local después del commit; no elimina una URL externa.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 404 id inexistente.

Endpoints 33-34

33 GET /api/preguntas

Listar todos los elementos del módulo para el personal.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → PreguntaController::index → acceso/validación → PreguntaService::listar → PreguntaRepository::listar → Pregunta::toArray/Response.

Persistencia: preguntas_frecuentes

Respuesta: 200, {preguntas:[...]}.

Reglas y efectos: Incluye elementos no publicados; no altera su estado.

Errores: 401 sin sesión válida; 403 con rol insuficiente.

34 POST /api/preguntas

Crear un elemento del módulo.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: pregunta (≤255) y respuesta, ambas no vacías.

Recorrido: public/index.php → PreguntaController::store → acceso/validación → PreguntaService::guardar → PreguntaRepository::crear → Pregunta::toArray/Response.

Persistencia: preguntas_frecuentes; historial_acciones (UsuarioRepository::registrar)

Respuesta: 201, {pregunta:{...}}.

Reglas y efectos: Transacción de datos más auditoría. Comienza NO_PUBLICADO.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 400 JSON inválido; 422 datos/archivos inválidos; 404 id inexistente en edición.

Endpoints 35-36

35 PUT /api/preguntas/{id}

Editar los datos del elemento existente.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: pregunta (≤ 255) y respuesta, ambas no vacías.

Recorrido: public/index.php → PreguntaController::update → acceso/validación → PreguntaService::guardar → PreguntaRepository::buscar/actualizar → Pregunta::toArray/Response.

Persistencia: preguntas_frecuentes; historial_acciones (UsuarioRepository::registrar)

Respuesta: 200, {pregunta:{...}}.

Reglas y efectos: Transacción de datos más auditoría. Conserva el estado de publicación existente.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 400 JSON inválido; 422 datos/archivos inválidos; 404 id inexistente en edición.

36 PATCH /api/preguntas/{id}/estado

Publicar o retirar el contenido mediante una operación separada.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: JSON: estado=PUBLICADO o NO_PUBLICADO.

Recorrido: public/index.php → PreguntaController::updateEstado → acceso/validación → PreguntaService::estado → PreguntaRepository::buscar/actualizar → Pregunta::toArray/Response.

Persistencia: preguntas_frecuentes; historial_acciones (UsuarioRepository::registrar)

Respuesta: 200, {pregunta:{...}}.

Reglas y efectos: Si cambia el estado, persiste y audita PUBLICAR/DESPUBLICAR en una transacción. Si ya tiene ese estado devuelve el elemento sin duplicar acción. No implementa aprobación de Directora.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 422 estado inválido; 404 id inexistente.

Endpoints 37-38

37 DELETE /api/preguntas/{id}

Eliminar un elemento y mantener constancia en historial.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → PreguntaController::destroy → acceso/validación → PreguntaService::eliminar → PreguntaRepository::buscar/eliminar → Pregunta::toArray/Response.

Persistencia: preguntas_frecuentes; historial_acciones (UsuarioRepository::registrar)

Respuesta: 204 sin cuerpo.

Reglas y efectos: El servicio registra ELIMINAR y borra en la misma transacción. No elimina el registro de auditoría.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 404 id inexistente.

38 GET /api/franjas

Listar franjas para administrarlas, también llenas o vencidas.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → ReservaController::franjas → acceso/validación → ReservaService::franjas → ReservaRepository::franjas → Franja/Reserva → Response.

Persistencia: franjas_disponibilidad y reservas

Respuesta: 200, {franjas:[{id,fecha,hora_inicio,hora_fin,tipo,cupos_totales,cupos_disponibles},...]}.

Reglas y efectos: El repositorio cuenta reservas por franja; la consulta pública filtra fecha desde hoy y cupos disponibles positivos.

Errores: 401 sin sesión válida; 403 con rol insuficiente.

Endpoints 39-40

39 POST /api/franjas

Crear una franja de agenda académica.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: fecha desde hoy; hora_inicio/hora_fin HH:MM con fin posterior; tipo PRUEBA_MANEJO, RENOVACION_NORMAL o RENOVACION_URGENTE; cupos_totales entero de 1 a 20.

Recorrido: public/index.php → ReservaController::crearFranja → acceso/validación → ReservaService::guardarFranja → ReservaRepository::franja con bloqueo/horarioExiste/crearFranja o actualizarFranja → Franja/Reserva → Response.

Persistencia: franjas_disponibilidad y reservas; historial_acciones

Respuesta: 201, {franja:{...}}.

Reglas y efectos: Transacción y auditoría. Horario/fecha/tipo únicos. En edición se bloquea la franja y no se reducen cupos por debajo de reservas existentes.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 422 fecha/horario/tipo/cupos/duplicación; 404 id no existente en edición.

40 PUT /api/franjas/{id}

Editar horario, tipo o capacidad de una franja académica.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: fecha desde hoy; hora_inicio/hora_fin HH:MM con fin posterior; tipo PRUEBA_MANEJO, RENOVACION_NORMAL o RENOVACION_URGENTE; cupos_totales entero de 1 a 20.

Recorrido: public/index.php → ReservaController::actualizarFranja → acceso/validación → ReservaService::guardarFranja → ReservaRepository::franja con bloqueo/horarioExiste/crearFranja o actualizarFranja → Franja/Reserva → Response.

Persistencia: franjas_disponibilidad y reservas; historial_acciones

Respuesta: 200, {franja:{...}}.

Reglas y efectos: Transacción y auditoría. Horario/fecha/tipo únicos. En edición se bloquea la franja y no se reducen cupos por debajo de reservas existentes.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 422 fecha/horario/tipo/cupos/duplicación; 404 id no existente en edición.

Endpoints 41-42

41 DELETE /api/franjas/{id}

Eliminar una franja que todavía no tiene reservas.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: Sin cuerpo. El id de ruta, cuando existe, es un entero positivo.

Recorrido: public/index.php → ReservaController::eliminarFranja → acceso/validación → ReservaService::eliminarFranja → ReservaRepository::franja con bloqueo/eliminarFranja → Franja/Reserva → Response.

Persistencia: franjas_disponibilidad y reservas; historial_acciones

Respuesta: 204 sin cuerpo.

Reglas y efectos: Transacción, bloqueo de franja y auditoría. Si ya tiene reservas, se rechaza en lugar de romper las referencias.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 404 id no existente; 422 franja con reservas.

42 GET /api/agenda

Consultar la agenda y su resumen para el personal.

Acceso: Bearer válido, PERSONAL_IMSJ y cuenta activa.

Entrada: Query vista=día|semana|mes y fecha=AAAA-MM-DD. Defaults: día actual en Uruguay.

Recorrido: public/index.php → ReservaController::agenda → acceso/validación → ReservaService::agenda → ReservaRepository::agenda → Franja/Reserva → Response.

Persistencia: franjas_disponibilidad y reservas

Respuesta: 200, {reservas:[...con cedula...], resumen:{total,urgentes}}.

Reglas y efectos: Día exacto; semana lunes-domingo; mes completo. SELECT por rango de fechas con usuario/franja. Urgentes cuenta RENOVACION_URGENTE.

Errores: 401 sin sesión válida; 403 con rol insuficiente. 422 vista/fecha inválida.

Transacciones y operación

Una reserva sobre el último cupo

1. Validar Bearer ciudadano e identificador de franja.
2. Iniciar transacción y bloquear la franja con SELECT FOR UPDATE.
3. Comprobar fecha, duplicación y contar reservas.
4. Si hay capacidad, insertar la reserva y confirmar.
5. La segunda petición espera el bloqueo; al continuar observa el cupo ocupado y recibe 422. La unicidad usuario/franja complementa esa comprobación.

Contenido con un archivo nuevo

1. Validar tipo, contenido y tamaño.
2. Guardar con nombre aleatorio y recordar qué archivo es nuevo.
3. Guardar contenido y auditoría dentro de la misma transacción.
4. Si falla: rollback y eliminar archivos nuevos.
5. Si confirma: limpiar los archivos anteriores reemplazados. La base no se confirma sin su auditoría.

Instalar o actualizar

Instalación nueva: configurar .env, iniciar servicios, importar schema si la base está vacía y crear la primera cuenta mediante CLI explícita. Instalación existente: respaldo de datos/archivos, ensayo sobre copia, comparación del esquema, importación no destructiva, mismo proyecto/volúmenes Compose y nuevo login. No se ejecutó esa transición sobre datos institucionales.

Soporte fuera del inventario API

GET /storage entrega recursos comprobados; OPTIONS prepara solicitudes CORS. No son nuevos módulos. El README del backend y SSOO documentan arranque y respaldo; Docker/Apache y recuperación siguen pendientes de ejecución.

Evidencia, fuentes y pendientes

Comprobaciones de la versión

La suite de integración recorre las 42 rutas sobre MySQL real y dos procesos PHP. Comprueba permisos, JSON, publicación, recursos válidos/inseguros, edición multipart del panel, sesiones, límite de intentos, último cupo concurrente y rollback de auditoría/archivo. La cifra final y hashes de archivos probados están en docs/verificacion.md y docs/evidencia_backend_nativo.json.

Falta completar

Construcción y arranque Docker/Apache; ensayo de actualización/restauración con copia de datos reales; recorrido visual completo de ambos frontends; revisión interna de Juan Robaina y aceptación académica de PHP nativo. CC-01 a CC-04 conservan sus definiciones pendientes; no se cierran por la reescritura.

IA: observaciones y suposiciones

Las explicaciones se derivan del código actual y sus consumidores. Ejemplos abreviados y valores de prueba son ilustrativos, no registros del cliente. No se suponen nuevas prioridades, fechas, categorías, Directora o campos de agenda. Las sugerencias documentales no acreditan acuerdos. El equipo registra aprobaciones del Inspector en actas; no se modificaron aquí.

Fuentes verificables

Referencia: RodrigoCazard/api-ejemplo-utu, carpeta api-simple; commit d6f61c999369b754d70bfa0621a154e9a630589a.

Abrir la referencia fijada en GitHub

Implementación: backend/public/index.php, controllers/, services/, repositories/, models/, core/ y database.sql. Documentación: flujo_endpoints_backend.md, api.md, referencia_api_simple.md, migracion_backend_vanilla.md y verificacion.md.

Autoría y revisión

Crédito del proyecto y su documentación: Tomás Cabrera, Juan Robaina, Gabriela Romero, Verónica Romero y Juan Corrales. Apoyo externo de IA identificado, sin crédito individual al asistente. Revisión interna y aceptación pendientes.